
FASE 1 · ANÁLISIS DE DISEÑO

Portal de Subcontratas OH Casas Modulares

| Informe técnico del diseño Stitch y propuesta de arquitectura de la aplicación móvil

Fuente analizada: stitch-export.zip (stitch_oh_casas_subcontratas_portal)

Sistema de diseño: Modular Architectural Luxury

Fecha: 22 de septiembre de 2026

Estado: Análisis y propuesta — sin implementación de código

0. Aviso sobre el acceso a Stitch MCP y Codex

En esta conversación de chat no hay ningún conector MCP de Stitch ni de Codex cargado. Se ha comprobado tanto el listado de herramientas activas como el directorio de conectores disponibles, y no aparece ningún servidor "Stitch" ni "Codex" — únicamente Claude Docs y Claude en Chrome estaban disponibles en el momento del análisis.

Esto puede deberse a que el conector esté activo en otra superficie (por ejemplo, Claude Code o Claude Desktop) y no se haya propagado a este chat concreto, o a que la conexión no llegara a activarse para esta conversación. Se recomienda verificar en **Ajustes → Conectores** que aparecen conectados en este mismo contexto.

Consecuencia práctica: todo este informe está basado al 100% en el análisis del archivo ZIP entregado, no en un contraste en vivo con el proyecto de Stitch.

1. Qué contiene exactamente el ZIP

Estructura real extraída del archivo:

```
stitch_oh_casas_subcontratas_portal/
├── obras_disponibles_oh_casas_modulares_premium_navy/
│   ├── code.html (471 líneas)
│   └── screen.png
├── detalle_y_postulaci_n_de_obra_premium_navy/
│   ├── code.html (424 líneas)
│   └── screen.png
├── club_oh_partner_puntos_y_recompensas_premium_navy/
│   ├── code.html (445 líneas)
│   └── screen.png
├── perfil_profesional_y_documentaci_n_premium_navy/
│   ├── code.html (452 líneas)
│   └── screen.png
├── perfil_administrador_oh_casas_modulares_premium/
│   ├── code.html (367 líneas)
│   └── screen.png
├── paleta_de_colores_y_gu_a_de_estilo_modular_luxury/
│   ├── code.html (514 líneas)
│   └── screen.png
└── modular_architectural_luxury/
    └── DESIGN.md (231 líneas — tokens en YAML + guía de marca)
```

Total: 2,8 MB. Seis exportaciones de pantalla (cada una HTML autocontenido + captura PNG) y un documento de sistema de diseño en Markdown/YAML. No hay ningún proyecto de código real (ni React, ni componentes reutilizables, ni build system): son prototipos estáticos HTML + Tailwind vía CDN, típicos de una exportación de Stitch.

2. Qué pantallas existen

Pantalla (carpeta)	Tipo	Bottom nav
Obras Disponibles	Listado principal (home del subcontratista)	Sí — 4 tabs
Detalle y Postulación de Obra	Detalle + formulario de oferta	No (top bar con "volver")
Club OH Partner	Puntos, nivel, privilegios, canje de recompensas	Sí — 4 tabs
Perfil Profesional y Documentación	Perfil del contratista + cumplimiento legal/PRL	Sí — 4 tabs
Perfil Administrador	Dashboard interno para el equipo de OH Casas	Sí — 4 tabs (distintos)
Paleta de colores y guía de estilo	No es pantalla de app — documentación visual generada por Stitch	No aplica

Importante: faltan pantallas que la propia navegación referencia pero que no vienen en el export (detalladas en el punto 4).

3. Qué componentes existen

Inventario de componentes repetidos de forma consistente entre pantallas:

- **Header superior** fijo (`sticky top-0`) con logo, badge de sección, notificaciones y avatar
- **Bottom navigation bar** de 4 pestañas (fijo, `fixed bottom-0`)
- **Tarjeta de obra/licitación**: imagen con gradiente + badges superpuestos, título, precio, chips de especificación, descripción, fila de plazo/requisitos, botones de acción
- **Banner de licitación prioritaria** con countdown y CTA destacado
- **Chips de filtro** horizontales con scroll
- **Grid de checklist de cumplimiento** (4 requisitos con check verde)
- **Formulario de postulación**: input monetario con sufijo, select, dropzone de archivos con lista de adjuntos, textarea, banner de bonus
- **Tarjeta hero de nivel/gamificación** (Club OH Partner): puntos, barra de progreso, racha
- **Grid de privilegios** (2x2) con icono + valor destacado + descripción
- **Lista de canje de recompensas** con precio en puntos y botón "Canjear"
- **Historial de puntos** (lista con icono, texto, delta de puntos)
- **Tarjeta de perfil** con foto, badge de verificación, métricas (rating, nº de obras)
- **Lista de documentación legal** con estado (vigente/vencido) y botón de descarga
- **Toggle switch** (modo supervisor / modo administrador)
- **KPI cards** tipo "bento grid" (panel admin): valor grande + label + micro-barra de progreso
- **Módulos de gestión operativa** (admin): icono + título + badge + sub-alerta accionable
- **Perspective switcher** tipo pill-toggle (Admin Gral. / Gremio-Subcontrata)
- **Banner de garantía de pago** (trust badge)

4. Qué navegación existe entre pantallas

El export contiene **dos bottom nav bars distintas**: dos "vistas de app" diferenciadas.

Vista Subcontratista

Obras ↔ Postulaciones ↔ Partner ↔ Perfil

Vista Administrador OH Casas

Obras ↔ Postulaciones ↔ Gremios ↔ Perfil Admin

Flujo detectado

- Obras Disponibles → (botón "Ver Licitación & Postular") → Detalle y Postulación de Obra (pantalla push, con botón "volver", sin tab bar)
- Perfil Profesional tiene un **toggle "Vista de Supervisor"** que cambia el texto entre "Modo Contratista Independiente" y "Modo Administrador OH Casas" (con JS real en el HTML)
- Perfil Administrador tiene un **selector de perspectiva** ("Admin Gral." / "Gremio-Subcontrata") con el mismo patrón

Esto sugiere que Stitch ha explorado un modelo de doble rol (subcontratista vs. personal interno de OH Casas) más que una app única sin roles.

Huecos de navegación

- Pantalla "Postulaciones" (listado de las propias ofertas enviadas) — no existe
- Pantalla "Gremios" (directorío de subcontratas, vista admin) — no existe
- Pantalla de gestión de "Obras" en modo admin (publicar/editar licitación) — sólo hay un botón CTA, no la pantalla de destino
- **No hay ninguna pantalla de login, registro, recuperación de contraseña ni onboarding** — el export asume sesión ya iniciada

5. Qué sistema de diseño se está utilizando

Nombre: **"Modular Architectural Luxury"**, documentado formalmente en `DESIGN.md` (frontmatter YAML + guía de marca en Markdown). Filosofía: brutalismo técnico + minimalismo de lujo — estética de plano arquitectónico

(blueprint), navys profundos, vidrio esmerilado, bordes "hairline".

Punto crítico para la implementación real: cada pantalla trae su propia configuración de Tailwind inline (`tailwind.config` distinto en cada script), con nombres de paleta diferentes entre sí (`architectural-950` , `brand.navy` , `navy.900` , tokens tipo `on-surface` ...). Los valores hexadecimales coinciden casi siempre, pero no hay un token system único y consistente aplicado en todas las pantallas — es una exportación de prototipos independientes, no un design system operativo.

6. La paleta de colores

Rol		Hex
Primary — Navy Arquitectónico Profundo		#0B192C
Secondary — Steel Navy		#1E3E62 / #426086
Tertiary — Precision Cobalt (acento interactivo)		#2563EB / #3B82F6
Accent — Terracotta técnico (avisos/urgencia)		#F25A2A
Gold (Club Partner / VIP)		#D4AF37
Success / Homologado		#10B981
Error		#BA1A1A
Technical Ivory (fondo)		#F8FAFC / #F8F9FF
Surface Blueprint		#F1F5F9 / #F1F3FC
Graphite / Slate (dark surfaces)		#0A0E14 / #121820
Borde estructural	—	rgba(11,25,44,0.08)

Semántica de estado ya definida (útil para mapear directamente a un sistema de theming): **Homologado/Conforme** = verde, **Partner Oro** = ámbar, **Informativo/Pliego técnico** = cobalto, **Urgente/Crítico** = terracota.

7. Tipografías

- **Familia única:** Plus Jakarta Sans (400/500/600/700/800), cargada vía Google Fonts en cada pantalla
- Excepciones puntuales: `JetBrains Mono` (sólo en la guía de estilo, para mostrar códigos hex) y `Material Symbols Outlined` como fuente de iconos (no tipografía de texto) en 2 de las 6 pantallas
- Escala completa definida en `DESIGN.md`: `display-xl` (56px/800) → `label-sm` (11px/700), con line-height y letter-spacing específicos por nivel, incluidas variantes mobile de display/headline

8. Espaciados, tamaños, bordes y radios

Radios (`rounded`): sm 2px · DEFAULT 4px · md 6px · lg 8px · xl 12px · full pill. En la práctica el HTML usa mucho `rounded-xl` / `rounded-2xl` de Tailwind estándar (12-16px) además de estos tokens — otra inconsistencia a resolver.

Espaciado: space-xs 4px · space-sm 8px · space-md 16px · space-lg 24px · space-xl 40px · gutters 16-24px · márgenes de canvas 20-48px según breakpoint.

Grid: 12 columnas desktop (≥1024px, contenedor máx. 1440px) / 8 columnas tablet / 4 columnas mobile.

Elevación: sombras multicapa en 3 niveles (reposo/hover/modal) + glassmorphism (`backdrop-filter: blur(16-24px)`) + bordes "hairline" de 1px en `rgba(11,25,44,0.08)` .

9. Iconos y assets

- **Dos sistemas de iconos distintos conviviendo** (inconsistencia a resolver):
 - 4 pantallas (Obras, Detalle, Club Partner, Perfil Profesional) → SVG inline estilo stroke (Feather/Heroicons-like, stroke-width 1.8-2, viewBox 24x24)
 - 2 pantallas (Perfil Administrador, Guía de Estilo) → Google Material Symbols Outlined (fuente de iconos)
- **Emojis decorativos** mezclados con los SVG (🏠 🏢 ✨ ⚡ 🌟 ★ € +) — funcionales en el prototipo pero no válidos

como sistema de iconografía de producción

- **Imágenes:** todas son URLs remotas de `googleusercontent.com` (placeholders generados por Google AI, dominio `lh3.googleusercontent.com/aida...`) — ninguna imagen real de OH Casas, ningún asset descargable en el ZIP más allá de los `screen.png` (capturas, no assets de producción)
- No hay logo vectorial real de OH Casas: en varias pantallas el "logo" es una caja con un signo "+" en CSS, no un SVG de marca

10. Estados de los componentes

Documentados explícitamente en la guía de estilo:

- Botones: reposo / hover / active (`active:scale-[0.99]`) / focus (ring de 1.5px cobalto + glow)
- Inputs: reposo / focus (borde+ring cobalto, sin outline nativo) — no hay estado de error visualmente definido en ningún formulario
- Checkboxes/radios: inactivo (borde hairline) / activo (relleno navy + check blanco)
- Badges semánticos: homologado (verde), urgente (naranja/rosa), informativo (azul), partner oro (ámbar)
- Toggle switch: on/off con transición (visto en Perfil Profesional)
- Tarjetas: reposo / hover (cambio de borde a cobalto + shadow)

No hay estados de carga (loading/skeleton), estados vacíos (empty state) ni estados de error de formulario en ninguna pantalla — habrá que diseñarlos, porque el diseño actual sólo cubre el "happy path".

11. Qué código/frontend ha generado Stitch

Cada `code.html` es una página HTML autocontenida: Tailwind CSS v3 vía CDN (`cdn.tailwindcss.com`) + config inline por script, Google Fonts vía link, y en 2 casos un pequeño script vanilla JS para interactividad simulada (el toggle de supervisor, el switch admin/gremio). No hay ningún framework de componentes (no React, no Vue, nada reutilizable como módulo), no hay build tooling, no hay lógica de negocio real, no hay llamadas a API — es maquetación estática con datos hardcoded ("Carlos Mendoza", "18.200 €", etc.).

12. Qué información adicional se puede obtener mediante Stitch MCP

Ninguna en esta conversación, por el motivo explicado en el punto 0. Si se confirma que el conector está activo en el entorno de trabajo, se podría:

- Contrastar si existen más pantallas en el proyecto de Stitch que no se exportaron en este ZIP (muy probable, dado que faltan Postulaciones, Gremios, Login/Registro)
- Verificar si Stitch guarda un token system central distinto al que aparece duplicado en cada HTML
- Comprobar si hay variantes o flujos adicionales (p. ej. estados de error, dark mode) que Stitch mantenga internamente

13. Qué partes del diseño son reutilizables como componentes

Alta prioridad / usados en casi todas las pantallas

Header BottomAppBar (2 variantes) Avatar con badge NotificationBell TenderCard Chip / FilterChip Badge semántico
PrimaryButton SecondaryButton GhostButton ProgressBar StatCard / KPICard

Media prioridad

ComplianceListItem FormInput FormSelect FormTextarea FileDropzone ToggleSwitch RewardListItem
HistoryListItem SectionHeader

Baja prioridad / muy específicos

HeroLevelCard AdminModuleCard ArchitecturalGridOverlay (decorativo)

Resumen de hallazgos clave antes de decidir el stack

1. **Faltan pantallas críticas** para una app funcional real: login, registro, recuperación de contraseña, onboarding, listado de postulaciones propias, listado de gremios/partners, gestión de obras en modo admin.
2. **Dos sistemas de iconos y varias configuraciones de Tailwind inconsistentes** entre pantallas — hay que normalizar antes de tokenizar en la app.
3. **No hay estados de carga, error ni vacío** diseñados — habrá que definirlos.
4. **No hay acceso confirmado a Stitch MCP ni a Codex en este chat** — el ZIP es la única fuente de verdad disponible ahora mismo.
5. El diseño ya "habla" de requisitos de seguridad reales (2FA forzado, registro de auditoría exportable, trazabilidad BIM/firma digital) — señal fuerte para la propuesta de seguridad backend.

Propuesta de arquitectura (Fase 2 — pendiente de decisión conjunta)

Nada de lo siguiente está implementado. Es una propuesta para decidir juntos antes de empezar por fases.

Arquitectura general de la app

App móvil nativa real con dos roles (Subcontratista / Administrador OH Casas) sobre la misma base de código, autenticación centralizada, backend propio con base de datos relacional, y el diseño de Stitch como fuente de verdad de la UI (tokenizado una sola vez, no duplicado por pantalla).

Stack tecnológico — recomendación

Capa	Recomendado	Alternativa	Por qué
Frontend móvil	React Native + Expo	Flutter	El diseño está en HTML/Tailwind; con NativeWind (Tailwind para RN) se pueden portar los tokens de DESIGN.md casi 1:1. Expo acelera build/distribución a Google Play (EAS Build)
Backend	NestJS (Node/TypeScript) + PostgreSQL + Prisma	Supabase (Postgres+Auth+Storage gestionado)	Datos sensibles B2B (pólizas RC, PRL, presupuestos, pagos): control total de auth, logs de auditoría y RLS a medida. Supabase es más rápido si se prioriza velocidad sobre control total
Auth	JWT (access + refresh) + 2FA (TOTP)	Supabase Auth / Auth0	El propio diseño admin exige "2FA forzado" — hay que construirlo desde el modelo de datos
Almacenamiento seguro en el móvil	expo-secure-store (Keychain/Keystore)	—	Nunca AsyncStorage plano para tokens
CI/CD y distribución	EAS Build + EAS Submit	Fastlane + GitHub Actions	Pipeline directo a Google Play (internal testing → producción)

Estructura de carpetas (propuesta)

```
oh-casas-subcontratas/  
├── apps/  
│   ├── mobile/ # React Native (Expo)  
│   │   ├── src/  
│   │   │   ├── app/ # navegación (expo-router o React Navigation)  
│   │   │   ├── components/ # componentes del punto 13, tokenizados  
│   │   │   ├── design-system/ # tokens (colores, tipografía, spacing) desde DESIGN.md  
│   │   │   ├── features/ # obras, postulaciones, perfil, club-partner, admin  
│   │   │   ├── services/ # clientes API, auth, storage seguro  
│   │   │   ├── hooks/  
│   │   │   ├── store/ # estado global (Zustand/Redux)  
│   │   └── app.config.ts  
│   └── api/ # NestJS  
│       ├── src/  
│       │   ├── auth/  
│       │   ├── users/  
│       │   ├── obras/  
│       │   ├── postulaciones/  
│       │   ├── club-partner/  
│       │   ├── documentos/  
│       │   ├── audit-log/  
│       │   └── common/ (guards, interceptors, filters)  
│       └── prisma/schema.prisma  
├── packages/  
│   └── shared-types/ # DTOs compartidos frontend/backend  
└── infra/ # IaC, docker-compose, scripts de deploy
```

Arquitectura frontend

- Patrón por features (no por tipo de archivo), con capa de `design-system/` que centraliza un único set de tokens (a diferencia del ZIP, que trae uno distinto por pantalla)
- Gestión de estado: Zustand o Redux Toolkit + TanStack Query para todo lo que venga de red (obras, postulaciones, puntos)
- Navegación: React Navigation (stack + bottom tabs anidados, replicando las dos bottom nav bars como dos tab navigators condicionados por rol)

Backend

NestJS modular por dominio (auth, obras, postulaciones, documentos, club-partner, admin), con:

- Guards de rol (Subcontratista / Admin / Superadmin — el diseño ya sugiere 3 niveles: "Admin Gral.", "Superadmin Nivel Central OH", "Gremio")
- Interceptor de auditoría (coincide con "Registro de auditoría exportable" del diseño admin)
- Rate limiting y validación estricta de entrada (class-validator)

Base de datos

PostgreSQL. Entidades principales: `usuarios`, `subcontratas`, `obras/licitaciones`, `postulaciones`, `documentos_homologacion` (con vencimientos), `puntos_club_partner`, `canjes`, `logs_auditoria`. Row Level Security si se migra a Supabase, o guards a nivel de servicio si se opta por NestJS puro.

Autenticación y gestión de sesiones

- Login con email/contraseña (Argon2 para hash) + 2FA TOTP opcional/forzado para rol admin (tal como pide el propio diseño)
- Access token de corta duración (15 min) + refresh token rotativo almacenado en expo-secure-store
- Recuperación de contraseña vía email con token de un solo uso y expiración corta
- Cierre de sesión con revocación de refresh token en backend (no sólo borrado local)

Almacenamiento seguro

- Tokens y credenciales: expo-secure-store (nunca AsyncStorage)
- Documentos sensibles (pólizas RC, certificados PRL): backend con storage cifrado en reposo (S3-compatible + cifrado del lado del servidor), nunca cacheados sin cifrar en el dispositivo

Navegación

Dos TabNavigator (Subcontratista: Obras/Postulaciones/Partner/Perfil — Admin: Obras/Postulaciones/Gremios/Perfil Admin) seleccionados por rol tras login, con un StackNavigator superior para pantallas push (detalle de obra, login, registro, recuperación).

Estrategia de testing

- Unit: Jest (lógica de negocio, servicios NestJS)
- Componentes: React Native Testing Library
- E2E: Detox o Maestro sobre el build de Expo
- Backend: tests de integración con base de datos de test (Testcontainers)
- Seguridad: revisión de dependencias (npm audit / Snyk), tests de autorización por rol

Estrategia de seguridad

- OWASP MASVS como referencia para la app móvil
- Certificate pinning en producción
- Ofuscación de build de producción (Hermes + ProGuard/R8 en Android)
- Auditoría de dependencias en CI

Codex como segundo agente: aquí es donde entraría la revisión de arquitectura, código, seguridad y tests. En cuanto se confirme que el plugin está realmente accesible en el entorno de trabajo, se integra como revisor en el flujo de PRs.

Build y distribución Android/Google Play

- EAS Build (perfil development → preview → production)
- Firma con keystore gestionado por Expo o propio
- Google Play Console: pista interna → cerrada → producción, con política de privacidad y declaración de datos (obligatoria dado que se manejan documentos legales/fiscales de terceros)

